Memora AI

Find the document you own but cannot find

Project report: what was built, how the system works, the technology and algorithms used, and what the measurements say.

Status as of	21 October 2026
Repository	94 commits on master, every retrieval change measured against a gold set
Headline result	Recall@1 0.842 (dense only) → 0.905 (two-stage + visual search, 25-row gold set); refusal accuracy 0.000 → 0.500
Source of every figure	`docs/RESULTS.md` and the run files it cites. Anything without a file behind it is marked.

1. What Memora is

You own the document and it is on one of your devices, but you cannot find it. An Aadhaar card, a marksheet, a fee receipt, a PDF a professor sent: each arrived through a different app, each app searches only its own silo, and most of these documents are photographs with no text layer, so keyword search returns nothing at all.

Memora accepts a human-shaped description of a document and returns the document. You remember what kind of thing it was, who sent it, roughly when; you do not remember `IMG_20240817_113244.jpg`.

Two layers

Layer	What it does	Success means
One — find the file	You describe it, Memora ranks your documents and shows thumbnails. No generative model is involved.	Correct file at **rank 1**. The answer is the file.
Two — ask about contents	A grounded answer with citations back into the source, built on layer one.	Every answer cites chunk IDs; weak evidence produces “I don't have that”, never a guess.

The seven hero queries (the acceptance tests)

#	Query	Why it is hard
1	find my Aadhaar card	A phone photo with no text layer; invisible without OCR.
2	the PDF a professor sent	Pure metadata: sender role, file type. Vector search alone returns noise.
3	the fee receipt from last semester	A vague time phrase must resolve to a date range first.
4	the screenshot with the database error	Remembered by how it looked; text exists only in pixels.
5	my 10th marksheet (three stored)	Disambiguation: the right one must be first.
6	find my passport (never stored)	Must refuse. Absence, not sensitivity, is the trigger.
7	the group photo	No text anywhere. Needs the image itself to be searchable.

2. How the whole system works

The architecture is a personal server (the laptop) plus a phone shell. All business logic lives in the FastAPI backend; the React app only renders and dispatches.

2.1 Components

Part	Technology	Role
API	FastAPI, Pydantic v2, SQLAlchemy 2.0 async, Alembic	All business logic, repositories, ingestion, search
Database	PostgreSQL 16 + pgvector, Row-Level Security	Documents, chunks, text vectors (768-d), image vectors, isolation
OCR service	Tesseract in a container	Reads text from photos and scanned pages
Visual search	CLIP ViT-B/32 in a container	Embeds images and text into one shared space
Text embeddings	gemini-embedding-001, truncated to 768 dims, L2-normalised	Dense vectors for chunks and queries
Stage 1 (query understanding)	Rule-based extractor (default for on-device); local qwen2.5:3b via Ollama and hosted Gemini kept as comparison arms	Turns a question into filters
Web app	React 19, Vite 6, TypeScript, Tailwind 4, shadcn/ui, PWA	Search UI, document viewer, chat, pairing
Native shell	Capacitor 8 (Android)	Scans files already on the phone
Shared code	packages/core (platform-neutral TypeScript, no DOM types)	Generated API types, platform interfaces
Cache / queue	Redis, arq	Provisioned; off the critical path after the product reframe

2.2 Ingestion: from a raw file to a searchable document

File (upload, or found by device scan)	Parse PDF text / OCR / image / .ics / .eml / chat .txt	**Mask** Aadhaar + PAN	Chunk ~400 tokens, 12% overlap	Embed text (Gemini) + image (CLIP)	Store rows, vectors, thumbnail

Step by step:

- **Parse.** PDFs go through PyMuPDF; images and scanned pages go through Tesseract OCR in a container. Calendar (.ics), email (.eml) and WhatsApp-style chat exports have dedicated parsers. A file with no extractable text (a plain photo, a laminated ID card) is no longer rejected: it is kept and becomes findable through CLIP.
- **Mask before anything leaves the machine.** Aadhaar (12 digits, Verhoeff-checked) and PAN (`ABCDE1234F`) are replaced with tokens before embedding *and* before storage. `MaskedText` is the only type the embedder accepts, verified on content rather than provenance, so a forged instance carrying an identifier is rejected. Scope is deliberately Aadhaar and PAN only; phone numbers match order numbers too, and masking those silently destroys retrievable text.
- **Chunk.** Structure-aware chunks of 300–500 tokens (target 400), 12% overlap, with a contextual prefix (document title plus section heading) prepended before embedding.
- **Label.** A rule-based classifier types each document (id card, marksheet, receipt, letter …) from its filename and extracted text. This label is what later powers refusal.
- **Embed.** Text chunks get 768-d vectors. Images additionally get a CLIP vector, stored with the model name per row so a model change is visible.
- **Store.** Original blob, thumbnail, chunk rows, vectors. Every user-owned table has Row-Level Security in the same migration that creates it.

2.3 Search: the two-stage retrieval contract

Query “the mail yash sent to rutuja”	**Stage 1** filters + semantic query	**Stage 2** dense search + visual fusion	Aggregate chunks → documents	Rank 1 file + thumbnail + snippet

- **Stage 1, query understanding.** Reads *only the user's own query text* and returns structured filters: document type, sender, source type (email / pdf / image / calendar …), date range, entities. In the current default this is a deterministic rule table, not a model. Filters are **soft**: a mismatching document is demoted by 0.08, not removed, because a wrong hard filter would delete the right answer.
- **Stage 2, retrieval.** Dense pgvector cosine search (HNSW index) fetches the nearest chunks, with soft filters applied. The fetch window is adaptive: if fewer documents surface than requested it doubles, up to 2000 chunks, because one calendar export holds 353 of 377 chunks and a fixed window starved the result list. Text and image result lists are then fused with **Reciprocal Rank Fusion, k = 60**: score = Σ 1/(60 + rank). *The design also specifies BM25 keyword search fused with dense; it is not implemented in the code (see section 6).*
- **Visual index, gated.** CLIP is consulted only for documents the text index did not find (see section 4.3 for why).
- **Aggregate to documents.** Each document is scored by its best chunk; two chunks of one PDF are one result. The best chunk is returned as the snippet.
- **Tie-break as a decision.** Exact RRF ties resolve by document-type agreement, then by each side's own-modality strength, never by comparing a CLIP score to a text score.
- **Refusal.** If the query names a document type the corpus does not hold, the system answers “not found” regardless of similarity.

Layer two (chat) reuses this retrieval, then asks a generation model to answer from the retrieved chunks. Retrieved text is delimited as data and the model is told to ignore instructions inside it; the response must return source chunk IDs, and no citation counts as a failed response.

2.4 Device scan and pairing

A PWA cannot enumerate a phone's Downloads folder or gallery; browsers sandbox storage on purpose. So the Capacitor Android shell does the scan and uploads what it finds. Pairing keeps the laptop's port from being open to anyone on the same hotspot:

- The laptop shows a **pairing code** as a QR: short-lived, single-use.
- The phone scans it and receives a **device token**, held only in the phone's secure storage; never shown, never in a URL or log.
- Every private endpoint requires the token. Minting a pairing code is an operator-only power, which closed a self-pairing bypass.
- Reconciliation: a file that disappears from the phone is marked “removed from device” rather than left as a broken result. Removal is computed only from a scan that declares it enumerated everything, because the first pass indexes only the newest 500 files and treating that as the whole would mark most of the corpus deleted.

2.5 Security and privacy model

Rule	How it is enforced
Users see only their own data	Repository layer injects a server-derived user_id; Postgres RLS is ON and FORCED in the same migration as each table
RLS is real, not decorative	API connects as `memora_app` (NOSUPERUSER, NOBYPASSRLS, owns nothing); `SET LOCAL` not SET; two-argument `current_setting` so a missing variable fails closed; tests use raw SQL with no WHERE clause. Visible live as `role` on /health/ready
No document content in any layer-one prompt	Structural test: layer one may not import the generation module, and the stage-1 prompt builder accepts a query string and nothing else
Identifiers masked first	Aadhaar and PAN masked before storage and before embedding
Retrieved text is data	Delimited in prompts; model told to ignore instructions inside it
Secrets	Config from environment only; .env blocked by pre-commit hook (verified by making it fire); tokens never logged
Deletion is demonstrable	Account/document erasure removes rows, vectors and files

3. Technology and algorithms

Algorithm / technique	Where used	Why this one
Dense vector search (cosine, pgvector HNSW)	Text chunks, 768-d	Meaning-based match; Postgres already holds the data, no separate vector DB
BM25 keyword search	**Specified, not built**	Would catch exact names and IDs that embeddings blur; future work
Adaptive fetch window	Dense retrieval	Doubles until enough documents surface (cap 2000 chunks); fixed windows broke list completeness
Reciprocal Rank Fusion, k = 60	Text list + image list	Combines rankers whose scores are on different scales by using ranks only
Soft metadata filters (+/– 0.08)	Stage 1 output applied in stage 2	A wrong filter demotes rather than deletes (measured: wrong filter is worse than none)
Rule-based query understanding	Stage 1	Equal to the 3B model on every gold row, 0 ms, no failure modes
Rule-based document typing, shared vocabulary	Ingest labels and query classification	One table for both sides; deterministic; powers refusal
CLIP ViT-B/32 image–text embeddings	Visual search	Finds photos with no text by description
Text-gated visual fusion	Image list admitted only for docs text missed	Fixes the non-selectivity of a nearest-neighbour index (section 4.3)
Structure-aware chunking with overlap	Ingest	300–500 tokens, 12% overlap, title + heading prefix
Max-chunk document scoring	Aggregation	A single decisive chunk should carry a document
Document-type agreement refusal	Absence detection	Score thresholds cannot detect absence (section 4.4)
Regex + Verhoeff checksum masking	Aadhaar / PAN	Detectable without guessing; checksum cuts false positives
OCR: Tesseract	Images and scans	PaddleOCR was tried and cannot run on this hardware (oneDNN crash)
Constrained JSON decoding	Local LLM arm	Removed the malformed-output failure class (0 failures in 23 queries)
Bearer tokens + pairing code	Phone to laptop	Short-lived QR code exchanged for a long-lived device token

Why not simpler or fancier

- **Not vector search alone.** Most personal queries are metadata queries in semantic clothing. Straight vector similarity fails hero queries 1, 2, 3 and 5 while looking good on generic content questions.
- **Not a separate vector database, LangGraph, GraphRAG or agentic loops.** pgvector is sufficient at this scale (Qdrant is the documented scale-out path); plain Python pipelines are easier to test and explain.
- **Not a language model at layer one.** Both a privacy rule and, as measured, unnecessary.

4. Results: what was measured

Recall@1 is the primary metric: the fraction of answerable queries whose expected file ranks first. The gold set started at 21 rows on 11 documents, grew to 23 rows on 14 documents when visual search made three text-free files ingestable, and is now 25 rows. **Numbers across those boundaries are not directly comparable.** One answerable row is about 0.05 of Recall@1, so read counts, not fractions.

4.1 The progression

Step	Recall@1	Note
Dense-only baseline (committed before query understanding existed)	0.842 (16/19)	Refusal 0/2. Three misses: two recovered later, one has no text
+ stage 1, hosted Gemini	0.947 (18/19)	Central result. Hand-transcribed; the only figure without a machine-written file
Wrong extractor (gemini-flash-lite)	0.789	Below the no-extractor baseline: a wrong filter is worse than none
+ stage 1, local qwen2.5:3b	0.895	No quota, reproducible, 23/23 valid extractions
+ CLIP, full coverage, text-gated	0.895 (17/19)	q019 (balcony photo) findable for the first time
+ refusal by document type	0.895; refusal 0.500	Recall@5 reaches 1.000; zero false refusals
+ RRF tie-break (20-row set)	0.950	Visual hits no longer lose ties by insertion order
Current, 25 rows (rules or qwen2.5:3b)	0.905 (R@5 1.000, MRR 0.952)	q025 added deliberately and fails; refusal 0.500

4.2 Stage 1 does not need a language model

Arm	Recall@1	MRR@10	Stage 1 latency	Failure modes
dense-only	0.842	—	—	—
hosted gemini-2.5-flash	0.947 (older corpus)	0.947	2–3 s	quota 20/day, model closed to new keys
local qwen2.5:3b	0.905	0.952	~4.6 s	model load 89 s cold; omits optional fields
rules, no model	**0.905**	**0.952**	**0 ms**	**none by construction**

Same outcome on every individual row; the per-row diff is empty. Two 3B models also scored identically while failing on *opposite* rows (one follows a counter-intuitive prompt rule but cannot do date arithmetic, the other the reverse), showing that an aggregate can hide which queries work. Caveat stated in the results: 25 rows is small and a word table fits the queries it was written beside.

4.3 Why adding visual search first broke ranking

RRF rewards a document for appearing in both lists, which is only sound when both lists are *selective*. Text retrieval is selective: a document unrelated to the query is simply not near it. A CLIP index is not, because cosine similarity is defined for every pair so it returns a nearest neighbour for every query. Documents carrying both OCR text and image vectors collected an agreement bonus on queries they had nothing to do with.

Configuration	Image vectors	Recall@1
No visual search	0 of 8	0.895
Raw RRF, partial coverage	3 of 8	0.895 (defect hidden)
Raw RRF, full coverage (filtered)	8 of 8	0.632
Raw RRF, full coverage (unfiltered, from a committed note)	8 of 8	0.316
Text-gated: image index adds only docs the text index missed	8 of 8	**0.895**

A threshold cannot fix it: the real match scored 0.348 while noise ran 0.22–0.28. The gate needs no constant. The gate then made the lists disjoint, so RRF ties became exact (1/61 each) and insertion order capped purely visual hits at rank 2; the tie-break rule above fixed that. One honest open problem remains: q025 (“rectangular plastic pass with my face printed on it”) still lands at rank 2, and is left failing on purpose so the headline does not overstate visual retrieval.

4.4 Refusal

No score threshold works: the absent “college fee receipt” scored 0.647, above nine correct answers, because the calendar genuinely mentions college and fees. Refusing by **document-type agreement** raised refusal accuracy from 0.000 to 0.500 with zero false refusals. The remaining 0.500 is the mechanism's ceiling: visual absence controls (“photo at the Taj Mahal”) name no document type, and no mechanism here can report that a picture is absent.

5. What we did, in order

Phase	What was done
1. Vertical slice	PDF → chunks → pgvector → grounded answer with chunk IDs; RLS shipped with the tables; PWA shell with mocked chat; corpus inspector
Product reframe	From generic chat to document retrieval: “the answer is the file.” Hero queries became acceptance tests
2. OCR + masking	Tesseract in a container; Aadhaar/PAN masking verified against forgery; PaddleOCR claim corrected after it proved unrunnable
Storage and /search	Uploaded blobs and thumbnails, demonstrable erasure; ranked-document /search with no generation call
Text ingestion	Calendar, email and chat parsers; adaptive fetch window; personal documents removed from git and the class of bug closed by a test
3. Gold set and baseline	Real 21-row gold set; dense-only baseline committed *before* query understanding existed
4. Query understanding	Two-stage retrieval with soft filters; Recall@1 0.842 → 0.947; quota budget measured and cached; stage-1 failures made visible
Ablations	Skip-heuristic measured and shown to cost nothing, then falsified with stage 1 forced on every row
Local stage 1	qwen2.5:3b and llama3.2:3b measured; local made the default because it is reproducible and keeps queries on the device
Search UI	Ranked results with snippet first, thumbnails, in-app viewer, understood-filter chips, degraded-mode banner
CLIP visual search	Images embedded; fusion failure diagnosed and fixed with the text gate; backfill for derived data
Refusal	Document-type agreement; refusal accuracy 0.000 → 0.500
Capacitor + pairing	Android shell, QR pairing, device token on every private route, consent screen, scan of files already on the phone, safe reconciliation
Fusion root-cause work	RRF preconditions named; tie-break decision; chunk damping measured and rejected; rank-1 loss diagnosed before fixing
Rules-only stage 1	Prediction written first, then measured: identical to the 3B model on every row. On-device build ships no LLM

Working method

- **Plan before code.** Each phase has a plan in `prompts/`, approved before implementation.
- **Pre-registered predictions.** Expected results (and which rows should move) were written down before each run. This caught a regression hidden behind an improved headline, and an inert fix whose diagnosis had been asserted rather than checked.
- **Baseline first.** The naive-retrieval number was committed before any improvement existed.
- **Four gates after every change:** pytest and ruff; web typecheck and build; Alembic upgrade; the gold-set evaluation. A test run taken while containers are down is not evidence, because database tests skip themselves.
- **Stop tuning retrieval.** Once the headline crossed its bar, remaining time went to capability gaps (visual search, device indexing) instead of a further-optimised number.

Lessons that generalise (from the measurements)

- A guard that only fires at 100% failure is not a guard; it now refuses runs above 25% stage-1 failures.
- An optimisation can expire silently: the skip heuristic was correct until refusal added a new signal it could not see.

- Listing a capability is not access to it: a model appeared in the API's list but returned 404 when called.
- A value that is set is not a value that is used: a shell variable silently overrode `.env`, so startup now prints which source won.
- A prediction can be right for the wrong reason; predict the quantity (the margin), not only the direction.

6. Limits, and what comes next

Known limits (stated so they are not discovered)

- **Small, single-owner corpus.** 14 documents, 25 gold rows, one person's archive. Findings about fusion, extractor substitution and refusal are mechanical rather than statistical.
- **One document is 93.6% of chunks** (353 of 377), which biases per-document scoring. The correction was measured, hurt results, and is disabled until a fairer corpus exists.
- **BM25 is not built.** The product design calls for dense + BM25 hybrid retrieval; the code does dense retrieval with soft filters and fuses only text with CLIP. Every number in this report was measured without BM25. A keyword layer is the obvious next retrieval addition.
- **Visual absence is unsolved** and **q025 fails**: purely visual retrieval can still be capped at rank 2.
- **Hosted arm not reproducible:** gemini-2.5-flash is closed to new API projects, so the 0.947 cannot be re-run by a new key. The 0.895/0.905 local and rules results can.
- **Device-scan path is not fully verified on hardware:** the Android shell and pairing commits are labelled as such in git history; treat on-phone behaviour as demonstrated only where a later commit says it was checked.
- **Auth is a pairing token**, not user accounts; a verified-token login is a later phase. Audio/Whisper and the Gmail connector are out of scope (documented future work).

Roadmap

Next	Why
Auth (replace the dev user id with a verified token)	RLS already enforces isolation; identity is the missing input
Larger, adversarial gold set	The one measurement that could overturn the rules-only result
Final report and UML	Ingestion and query sequence diagrams carry the weight
On-device build	ML Kit OCR, quantised ONNX CLIP (~50 MB), SQLite + sqlite-vec, ONNX MiniLM, rules for stage 1. RLS becomes vacuous on a single-user device and the report must say so
Future work, deliberately parked	Image captioning, handwriting OCR, audio search, Gmail connector, reranking and calibration

Where to look: `docs/PRODUCT.md` (what “working” means), `docs/RESULTS.md` (every number and its source file), `docs/ROADMAP.md`, `prompts/` (the plan behind each change).